Introduction to Haskell Programming

Computation as rewriting

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Computation as rewriting

- Use definitions to simplify expressions till no further simplification is possible

Computation as rewriting

- Use definitions to simplify expressions till no further simplification is possible
- An “answer” is an expression that cannot be further simplified

Computation as rewriting

- Use definitions to simplify expressions till no further simplification is possible
- An “answer” is an expression that cannot be further simplified
- Built-in simplifications

```
3+5              ⟶ 8
True || False    ⟶ True
```

Computation as rewriting

- Simplifications based on user-defined functions

```
power :: Int → Int → Int
power x 0 = 1
power x n = x * power x (n-1)
```

Computation as rewriting

```
power 3 2
⟶ 3 * power 3 (2-1)          user definition
                             (*) requires both operands to be numbers,
                             so power 3 (2-1) has to be simplified.
                             power does pattern matching on the second argument,
                             so (2-1) has to be simplified.
⟶ 3 * power 3 1              subtraction
⟶ 3 * (3 * power 3 (1-1))    user definition
⟶ 3 * (3 * power 3 0)        subtraction
⟶ 3 * (3 * 1)                user definition
⟶ 3 * 3                      multiplication
⟶ 9                          multiplication
```

Leftmost outermost reduction

- Identify the leftmost subexpression that qualifies for simplification ...

Leftmost outermost reduction

- Identify the leftmost subexpression that qualifies for simplification …
- …such that no expression containing it qualifies for simplification …

Leftmost outermost reduction

- Identify the leftmost subexpression that qualifies for simplification …
- …such that no expression containing it qualifies for simplification …
- …and simplify it!

Leftmost outermost reduction

- Identify the leftmost subexpression that qualifies for simplification ...
- ...such that no expression containing it qualifies for simplification ...
- ...and simplify it!
- For an expression of the form `e1 op e2` where `op` is a built-in operator ...

Leftmost outermost reduction

- Identify the leftmost subexpression that qualifies for simplification ...
- ...such that no expression containing it qualifies for simplification ...
- ...and simplify it!
- For an expression of the form `e1 op e2` where `op` is a built-in operator ...
- ...simplify `e1` first and then `e2` till they are values ...

Leftmost outermost reduction

- Identify the leftmost subexpression that qualifies for simplification ...
- ...such that no expression containing it qualifies for simplification ...
- ...and simplify it!
- For an expression of the form `e1 op e2` where `op` is a built-in operator ...
- ...simplify `e1` first and then `e2` till they are values ...
- ...like the number `5` instead of the expression `3+2`

No repeated computation

- Consider the following simplification sequence (not followed by Haskell)

```
power (4-2) (3-1)
⟶ power 2 (3-1)
⟶ power 2 2
⟶ 2 * power 2 (2-1)
⟶ 2 * power 2 1
⟶ 2 * (2 * power 2 (1-1))
⟶ 2 * (2 * power 2 0)
⟶ 2 * (2 * 1)
⟶ 2 * 2
⟶ 4
```

No repeated computation

- Consider the following simplification sequence (not followed by Haskell)

```
power (4-2) (3-1)
⟶ power 2 (3-1)
⟶ power 2 2
⟶ 2 * power 2 (2-1)
⟶ 2 * power 2 1
⟶ 2 * (2 * power 2 (1-1))
⟶ 2 * (2 * power 2 0)
⟶ 2 * (2 * 1)
⟶ 2 * 2
⟶ 4
```

- In the above, `4-2` is evaluated once, and `2` is used in the rest of the computation

No repeated computation

- It appears that Haskell has to repeat the computation of `4-2`, as follows:

```
power (4-2) (3-1)
⟶ power (4-2) (3-1)
⟶ power (4-2) 2
⟶ (4-2) * power (4-2) (2-1)
⟶ 2 * power (4-2) (2-1)
⟶ 2 * power (4-2) 1
⟶ 2 * ((4-2) * power (4-2) (1-1))
⟶ 2 * (2 * power (4-2) (1-1))
⟶ 2 * (2 * power (4-2) 0)
⟶ 2 * (2 * 1)
⟶ 2 * 2
⟶ 4
```

No repeated computation

- But Haskell remembers that `4-2` is a replacement for `x`

No repeated computation

- But Haskell remembers that `4-2` is a replacement for `x`
- Once an `e` occurring in place of `x` is simplified ...

No repeated computation

- But Haskell remembers that `4-2` is a replacement for `x`
- Once an `e` occurring in place of `x` is simplified …
- …all other instances of `e` occurring in place of `x` are automatically simplified

```
power (4-2) (3-1)
⟶ power (4-2) (3-1)
⟶ power (4-2) 2
⟶ (4-2) * power (4-2) (2-1)
⟶ 2 * power 2 (2-1)
⟶ 2 * power 2 1
⟶ 2 * (2 * power 2 (1-1))
⟶ 2 * (2 * power 2 0)
⟶ 2 * (2 * 1)                ⟶ 2 * 2              ⟶ 4
```

No repeated computation

- But Haskell remembers that `4-2` is a replacement for `x`
- Once an `e` occurring in place of `x` is simplified …
- …all other instances of `e` occurring in place of `x` are automatically simplified

```
power (4-2) (3-1)
⟶ power (4-2) (3-1)
⟶ power (4-2) 2
⟶ (4-2) * power (4-2) (2-1)
⟶ 2 * power 2 (2-1)
⟶ 2 * power 2 1
⟶ 2 * (2 * power 2 (1-1))
⟶ 2 * (2 * power 2 0)
⟶ 2 * (2 * 1)                ⟶ 2 * 2              ⟶ 4
```

- Internally, Haskell uses some pointers to achieve this!

No repeated computation

- The same is the case when one uses a named expression

```
s = 4-2 :: Int

power s (3-1)
⟶ power s (3-1)
⟶ power s 2
⟶ s * power s (2-1)
⟶ 2 * power 2 (2-1)
⟶ 2 * power 2 1
⟶ 2 * (2 * power 2 (1-1))
⟶ 2 * (2 * power 2 0)
⟶ 2 * (2 * 1)                ⟶ 2 * 2              ⟶ 4
```

Order of evaluation

- Two ways of computing `(8+3) * (5-3)`

```
(8+3)*(5-3) ⟶ 11*(5-3) ⟶ 11*2 ⟶ 22
(8+3)*(5-3) ⟶ (8+3)*2 ⟶ 11*2 ⟶ 22
```

Order of evaluation

- Two ways of computing `(8+3) * (5-3)`

```
(8+3)*(5-3) ⟶ 11*(5-3) ⟶ 11*2 ⟶ 22
(8+3)*(5-3) ⟶ (8+3)*2 ⟶ 11*2 ⟶ 22
```

- Haskell chooses the first option, since both operands of `(*)` need to be numbers, and `8+3` is leftmost

Order of evaluation

- Two ways of computing `(8+3) * (5-3)`

```
(8+3)*(5-3) ⟶ 11*(5-3) ⟶ 11*2 ⟶ 22
(8+3)*(5-3) ⟶ (8+3)*2 ⟶ 11*2 ⟶ 22
```

- Haskell chooses the first option, since both operands of `(*)` need to be numbers, and `8+3` is leftmost
- Two ways of computing `power (5+2) (4-4)`

```
power (5+2) (4-4) ⟶ power 7 (4-4) ⟶ power 7 0 ⟶ 1
power (5+2) (4-4) ⟶ power (5+2) 0 ⟶ 1
```

Order of evaluation

- Two ways of computing `(8+3) * (5-3)`

```
(8+3)*(5-3) ⟶ 11*(5-3) ⟶ 11*2 ⟶ 22
(8+3)*(5-3) ⟶ (8+3)*2 ⟶ 11*2 ⟶ 22
```

- Haskell chooses the first option, since both operands of `(*)` need to be numbers, and `8+3` is leftmost
- Two ways of computing `power (5+2) (4-4)`

```
power (5+2) (4-4) ⟶ power 7 (4-4) ⟶ power 7 0 ⟶ 1
power (5+2) (4-4) ⟶ power (5+2) 0 ⟶ 1
```

- Haskell chooses the second option, since `power` is user-defined

Order of evaluation

- Two ways of computing `(8+3) * (5-3)`

```
(8+3)*(5-3) ⟶ 11*(5-3) ⟶ 11*2 ⟶ 22
(8+3)*(5-3) ⟶ (8+3)*2 ⟶ 11*2 ⟶ 22
```

- Haskell chooses the first option, since both operands of `(*)` need to be numbers, and `8+3` is leftmost
- Two ways of computing `power (5+2) (4-4)`

```
power (5+2) (4-4) ⟶ power 7 (4-4) ⟶ power 7 0 ⟶ 1
power (5+2) (4-4) ⟶ power (5+2) 0 ⟶ 1
```

- Haskell chooses the second option, since `power` is user-defined
- The arguments are evaluated only based on need

Order of evaluation

- Two ways of computing `(8+3) * (5-3)`

```
(8+3)*(5-3) ⟶ 11*(5-3) ⟶ 11*2 ⟶ 22
(8+3)*(5-3) ⟶ (8+3)*2 ⟶ 11*2 ⟶ 22
```

- Haskell chooses the first option, since both operands of `(*)` need to be numbers, and `8+3` is leftmost
- Two ways of computing `power (5+2) (4-4)`

```
power (5+2) (4-4) ⟶ power 7 (4-4) ⟶ power 7 0 ⟶ 1
power (5+2) (4-4) ⟶ power (5+2) 0 ⟶ 1
```

- Haskell chooses the second option, since `power` is user-defined
- The arguments are evaluated only based on need
- The second argument needs to be evaluated to test if it is `0` or not

Lazy evaluation

- Any Haskell expression is of the form `f e`

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function
  - `e` is the expression to which it is applied.

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function
  - `e` is the expression to which it is applied.
- In `head (2:reverse [1..5])`

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function
  - `e` is the expression to which it is applied.
- In `head (2:reverse [1..5])`
  - `f` is `head`

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function
  - `e` is the expression to which it is applied.
- In `head (2:reverse [1..5])`
  - `f` is `head`
  - `e` is `2:reverse [1..5]`

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function
  - `e` is the expression to which it is applied.
- In `head (2:reverse [1..5])`
  - `f` is `head`
  - `e` is `2:reverse [1..5]`
- When `f` is a function name, Haskell reduces `f e` using the definition of `f`

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function
  - `e` is the expression to which it is applied.
- In `head (2:reverse [1..5])`
  - `f` is `head`
  - `e` is `2:reverse [1..5]`
- When `f` is a function name, Haskell reduces `f e` using the definition of `f`
- It is also possible that `f` is an expression that has a function type

Lazy evaluation

- Any Haskell expression is of the form `f e`
  - `f` is the outermost function
  - `e` is the expression to which it is applied.
- In `head (2:reverse [1..5])`
  - `f` is `head`
  - `e` is `2:reverse [1..5]`
- When `f` is a function name, Haskell reduces `f e` using the definition of `f`
- It is also possible that `f` is an expression that has a function type
- In such cases, the expression `f` will be evaluated to a function name (or a built-in operator)

Lazy evaluation

- The argument is not evaluated if the function definition does not force it to be evaluated

Lazy evaluation

- The argument is not evaluated if the function definition does not force it to be evaluated
- `head (2:reverse [1..5])` ⟶ `2`

Lazy evaluation

- The argument is not evaluated if the function definition does not force it to be evaluated
- `head (2:reverse [1..5]) ⟶ 2`
- Argument is evaluated if needed

```
last (2: reverse [1..3])
⟶ last (reverse [1..3])
⟶ last (revInto [] [1..3])
⟶ last (revInto [1] [2..3])
⟶ last (revInto [2,1] [3])
⟶ last (revInto [3,2,1] [])
⟶ last [3,2,1] ⟶ last [2,1] ⟶ last [1] ⟶ 1
```

Lazy evaluation

- What would `power (3 `div` 0) 0` return?

```
power :: Int → Int → Int
power x 0 = 1
power x n = x * power x (n-1)
```

Lazy evaluation

- What would `power (3 `div` 0) 0` return?

```
power :: Int → Int → Int
power x 0 = 1
power x n = x * power x (n-1)
```

- First definition ignores value of `x`

Lazy evaluation

- What would `power (3 `div` 0) 0` return?

```
power :: Int → Int → Int
power x 0 = 1
power x n = x * power x (n-1)
```

- First definition ignores value of `x`
- `power (3 `div` 0) 0` returns `1`

Lazy evaluation

- If all simplifications are possible, order of evaluation does not matter, same answer

Lazy evaluation

- If all simplifications are possible, order of evaluation does not matter, same answer
- One order may terminate, another may not

Lazy evaluation

- If all simplifications are possible, order of evaluation does not matter, same answer
- One order may terminate, another may not
- Lazy evaluation expands arguments by **need**

Lazy evaluation

- If all simplifications are possible, order of evaluation does not matter, same answer
- One order may terminate, another may not
- Lazy evaluation expands arguments by **need**
- Can terminate with an undefined sub-expression if that expression is not used

Lazy evaluation

- If all simplifications are possible, order of evaluation does not matter, same answer
- One order may terminate, another may not
- Lazy evaluation expands arguments by **need**
- Can terminate with an undefined sub-expression if that expression is not used
- What would be the result of the following?

```
length (map (`div` 0) [0..9])
```

Lazy evaluation

```
xs = map (+1) [0..3] :: [Int]
length xs
⟶                  length (map (+1) [0..3])
⟶                  length (0+1: map (+1) [1..3])
⟶ 1 +              length (map (+1) [1..3])
⟶ 1 +              length (1+1: map (+1) [2..3])
⟶ 1 + 1 +          length (map (+1) [2..3])
⟶ 1 + 1 +          length (2+1: map (+1) [3])
⟶ 1 + 1 + 1 +      length (map (+1) [3])
⟶ 1 + 1 + 1 +      length (3+1: map (+1) [])
⟶ 1 + 1 + 1 + 1 + length (map (+1) [])
⟶ 1 + 1 + 1 + 1 + length []
⟶ 1 + 1 + 1 + 1 + 0 ⟶ 4          -- xs is never fully evaluated!
```

Infinite lists

```
infList :: [Integer]
infList = infFrom 0
infFrom :: Integer → [Integer]
infFrom n = n: infFrom (n+1)

infList ⟶ [0,1,2,3,4,5,6,7,8,9,10,11,12, ... ]

head infList
⟶ head (infFrom 0)
⟶ head (0:infFrom (0+1))
⟶ 0
```

Infinite lists

```
infList = infFrom 0
infFrom n = n: infFrom (n+1)

take 2 infList
⟶ take 2 (infFrom 0)
⟶ take 2 (0:infFrom (0+1))
⟶ 0:take 1 (infFrom (0+1))
⟶ 0:take 1 ((0+1):infFrom ((0+1)+1))
⟶ 0:(0+1):take 0 (infFrom ((0+1)+1))
⟶ 0:(0+1):[]
⟶ 0:1:[]
```

Infinite lists

- Range notation extends to infinite lists

Infinite lists

- Range notation extends to infinite lists
- `[m..] = [m, m+1, m+2, ...]`

Infinite lists

- Range notation extends to infinite lists
- `[m..] = [m, m+1, m+2, ...]`
- `[m, m+d..] = [m, m+d, m+2d, m+3d, ...]`

Infinite lists

- Range notation extends to infinite lists
- `[m..] = [m, m+1, m+2, ...]`
- `[m, m+d..] = [m, m+d, m+2d, m+3d, ...]`
- Using infinite lists often simplifies programs

```
indexOf x l          = go x (zip [0..] l) where
    go x []          = -1
    go x ((i,y):ps) = if x == y then i else go x ps
```

Generating primes

- Start with the infinite list `[2,3,4, ... ]`

Generating primes

- Start with the infinite list `[2,3,4, ... ]`
- The head is a prime

Generating primes

- Start with the infinite list `[2,3,4, ... ]`
- The head is a prime
- Remove all its multiples from the tail and recursively compute primes

Generating primes

- Start with the infinite list `[2,3,4, ... ]`
- The head is a prime
- Remove all its multiples from the tail and recursively compute primes
- Haskell program:

```
primes = sieve [2..] where
    sieve (p:xs) = p:sieve [x | x ← xs, x `mod` p /= 0]
```

Generating primes

- Start with the infinite list `[2,3,4, ... ]`
- The head is a prime
- Remove all its multiples from the tail and recursively compute primes
- Haskell program:

```
primes = sieve [2..] where
    sieve (p:xs) = p:sieve [x | x ← xs, x `mod` p /= 0]
```

- The nth prime is `primes !! (n-1)`

Generating primes

```
primes
⟶ sieve [2..]
⟶ 2:sieve [x | x ← [3..], x `mod` 2 /= 0]
⟶ 2:sieve (3:[x | x ← [4..], x `mod` 2 /= 0])
⟶ 2:3:sieve [y | y ← [x | x ← [4..], x `mod` 2 /= 0], y `mod` 3 /= 0]
⟶ 2:3:sieve [y | y ← [x | x ← [5..], x `mod` 2 /= 0], y `mod` 3 /= 0]
⟶ 2:3:sieve [y | y ← 5:[x | x ← [6..], x `mod` 2 /= 0], y `mod` 3 /= 0]
⟶ 2:3:sieve (5:[y | y ← [x | x ← [6..], x `mod` 2 /= 0],
                                        y `mod` 3 /= 0])
⟶ 2:3:5:sieve [z ← [y | y ← [x | x ← [6..], x `mod` 2 /= 0],
                                              y `mod` 3 /= 0],
                                              z `mod` 5 /= 0]
⟶ ...
```